Trees, random forests, gradient boosting

Workflow labs use the variant template: Goal → Approach → Execution → Check → Report.

Learning objectives

- Fit a single decision tree, a random forest, and a gradient-boosted model on the same data, and compare their error.
- Read variable-importance output critically and know its biases.
- Explain when a tree ensemble is preferable to a linear model and when it is not.

Prerequisites

Cross-validation; regularisation.

Background

Tree-based models divide feature space by axis-aligned splits and fit a constant within each region. A single tree is interpretable but high variance; averaging many trees with resampling and random feature subsets gives random forests, which are among the strongest off-the-shelf methods on tabular data. Gradient boosting takes a different route: it fits trees sequentially, each correcting the residuals of the last, and usually wins on tabular benchmarks when carefully tuned.

Tree methods make few assumptions about the functional form of the response and handle interactions automatically. Their limitations are extrapolation (flat beyond the training range), variable-importance measures that can be misleading when features are correlated, and a general tendency to look better than they generalise unless honest resampling is used.

When reporting a random forest or gradient-boosted model, report the tuning procedure, the out-of-bag or CV error, the variable-importance method, and — if the goal is scientific — a permutation-based importance to cross-check the default Gini/gain importance.

Setup

```
library(tidyverse)
library(rpart)
library(ranger)
library(xgboost)
set.seed(42)
theme_set(theme_minimal(base_size = 12))
```

1. Goal

Predict `mpg` from `mtcars` with a single tree, a random forest, and gradient boosting; compare errors and importances.

2. Approach

```r
ggplot(d, aes(wt, mpg, colour = factor(cyl))) +
  geom_point() + labs(colour = "cyl")
```

3. Execution

```r
rf_fit   <- ranger(mpg ~ ., data = d, importance = "permutation",
                   num.trees = 500)
xg_fit   <- xgboost(
  data = as.matrix(d[, -1]), label = d$mpg,
  nrounds = 100, eta = 0.05, max_depth = 3,
  objective = "reg:squarederror", verbose = 0
)
```

4. Check

Honest error: CV for the tree and xgboost, OOB for ranger.

```r
folds <- sample(rep(1:K, length.out = nrow(d)))
mse <- function(y, yhat) mean((y - yhat)^2)

get_err <- function() {
  err_tree <- err_rf <- err_xg <- numeric(K)
  for (k in 1:K) {
    tr <- folds != k; te <- folds == k
    f_tr <- d[tr, ]; f_te <- d[te, ]
    t1 <- rpart(mpg ~ ., data = f_tr, cp = 0.01)
    r1 <- ranger(mpg ~ ., data = f_tr, num.trees = 500)
    x1 <- xgboost(
      data = as.matrix(f_tr[, -1]), label = f_tr$mpg,
      nrounds = 100, eta = 0.05, max_depth = 3,
      objective = "reg:squarederror", verbose = 0
    )
    err_tree[k] <- mse(f_te$mpg, predict(t1, f_te))
    err_rf[k]   <- mse(f_te$mpg, predict(r1, f_te)$predictions)
    err_xg[k]   <- mse(f_te$mpg, predict(x1, as.matrix(f_te[, -1])))
  }
  c(tree = mean(err_tree), rf = mean(err_rf), xgb = mean(err_xg))
}
errs <- get_err()
errs
```

```r
       imp = as.numeric(rf_fit$variable.importance)) |>
  arrange(desc(imp)) |>
  ggplot(aes(reorder(var, imp), imp)) + geom_col() +
  coord_flip() + labs(x = NULL, y = "permutation importance")
```

5. Report

> On `mtcars` ($n = 32$), 5-fold CV MSEs were `round(errs["tree"], 2)` (single tree), `round(errs["rf"], 2)` (random forest), and `round(errs["xgb"], 2)` (gradient boosting). Permutation importance from the random forest identified weight and displacement as the dominant predictors.

The CV errors are close because the sample is tiny and the problem is close to linear. On larger tabular

problems the gap opens and gradient boosting typically wins after tuning; but the tiny-sample regime is where trees and boosting most often look like magic and almost always lie.

Common pitfalls

- Reporting the training error of a forest; always use OOB or CV.
- Trusting Gini/gain importance for correlated predictors.
- Letting xgboost overfit with low `eta` and high `nrounds` without early stopping.

Further reading

- Breiman L (2001), *Random forests*.
- Chen T, Guestrin C (2016), *XGBoost: A scalable tree boosting system*.

Session info

See also — chapter index

- Methods in this chapter
- Find your method (Chapter 0)